UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Kleanthis Vasileiadis*
sdi: 1115202200017

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Fall Semester 2023*

Contents

1 Abstract **2**

2 Data processing and analysis **2**
2.1 Pre-processing . . . 2
2.2 Analysis . . . 3
2.3 Data partitioning for train, test and validation . . . 3
2.4 Vectorization . . . 4

3 Algorithms and Experiments **5**
3.1 Experiments . . . 5
3.1.1 Experiment 1 - Baseline models . . . 5
3.1.2 Experiment 2 - Number of epochs . . . 6
3.1.3 Experiment 3 - Learning rate . . . 8
3.1.4 Experiment 4 - Batch size . . . 9
3.1.5 Table of trials . . . 11
3.2 Hyper-parameter tuning . . . 11
3.2.1 Key Hyperparameters . . . 12
3.2.2 Hyperparameters Tuning Results . . . 12
3.2.3 Prevention of overfitting and underfitting . . . 13
3.3 Optimization techniques . . . 13
3.4 Evaluation . . . 14
3.4.1 ROC curve . . . 14
3.4.2 Learning Curve . . . 16
3.4.3 Classification Report . . . 17
3.4.4 Confusion matrix . . . 18

4 Results and Overall Analysis **20**
4.1 Results Analysis . . . 20
4.1.1 Best trial . . . 21
4.2 Comparison with the First Project . . . 22
4.3 Comparison with the second project . . . 23

5 Bibliography **23**

1. Abstract

This project is a continuation of Projects 1 and 2 and aims to develop a sentiment classification model using machine learning and artificial intelligence techniques in Python, applied to a given English-language Twitter dataset. The dataset remains the same as in the previous projects and consists of three main components: an ID that uniquely identifies each tweet, the text containing the tweet's content, and a sentiment label, where a value of 1 indicates a positive sentiment and 0 indicates a negative one. The goal of the classification process is to assign each tweet to one of these two sentiment categories.

In Project 1, classification was performed using the TF-IDF vectorization method in combination with a logistic regression machine learning model. In Project 2, Word2Vec embeddings were used as input to a feedforward neural network. In the current project, transformer-based neural networks will be employed instead. Specifically, fine-tuning will be performed on two pre-trained language models:

- **BERT** (notebook link)
- **DistilBERT** (notebook link)

The performance of these fine-tuned models will then be compared to the results obtained in Projects 1 and 2.

2. Data processing and analysis

2.1. Pre-processing

Pre-processing is a crucial step in machine learning and AI applications, as it ensures that the input data is clean, consistent, and suitable for training a sentiment classification model. In this project, the preprocessing pipeline is partially based on the techniques used in the previous projects. Some of the original preprocessing steps were retained, others were removed, and new techniques were introduced to better align with the requirements of transformer based models. This hybrid approach allowed for improved compatibility and flexibility, while still keeping the strengths of earlier methods.
The text preprocessing techniques employed are briefly described below:

- **Common techniques across both models:**
  - **Replacing emojis:** Converts emojis into their corresponding word meanings.
  - **Replacing emoticons:** Converts emoticons into their corresponding textual representations.
  - **Replacing HTML entities:** Translates HTML entities into readable text or the appropriate special characters.
  - **Replacing links:** Replaces URLs (starting with "http", "https", or "www") with the word "link".
  - **Fixing contractions:** Expands contractions (e.g., "don't" to "do not") to their full forms.

- **Techniques exclusive to BERT:**
  - **Replacing slang:** Translates informal or slang expressions into their standard language equivalents.
- **Techniques exclusive to DistilBERT:**
  - **Replacing mentions:** Replaces user mentions (e.g., "@username") with the placeholder word "mention".

The process of replacing emoticons, HTML entities, and slang was carried out using custom lists and dictionaries, which were carefully constructed through analysis of the training dataset. The replacement of emojis was performed using the emoji library. All other replacements were implemented using regular expressions via the re library. Finally, contraction expansion was handled using the contractions library.

2.2. Analysis

Before proceeding with model training, it is essential to analyze and visualize the dataset to gain insights into its structure and characteristics. This step involves exploring the raw text data, identifying patterns, and understanding the distribution of words and sentences. By performing this analysis both before and after preprocessing, we can evaluate the impact of text cleaning techniques and ensure that important information is preserved while noise is reduced.

Since the same dataset was utilized as in the previous projects, an in-depth exploratory data analysis will not be repeated in this report. Nonetheless, it is important to highlight several key characteristics of the dataset:

- The dataset contains a total of 211,983 tweets. Of these, approximently 191K tweets are labeled.
- The labeled portion of the dataset is balanced, with an approximately equal number of positive and negative samples, which contributes to unbiased model training and evaluation.
- There are no missing values in any of the dataset's components, ensuring data completeness.

2.3. Data partitioning for train, test and validation

As in the previous projects, the dataset was provided pre-partitioned into training, validation, and test sets. The distribution follows a standard and well-balanced ratio: approximately 70 percent of the data (149K tweets) is used for training, 20 percent (42K tweets) for validation, and 10 percent (22K tweets) for testing. This split ensures a well balanced approach, allowing the model to learn effectively from a large portion of the data while keeping sufficient samples for hyperparameter tuning and evaluation. The validation set helps in preventing overfitting by guiding model adjustments, while the independent test set provides a final assessment of performance. These ratios are widely used in deep learning tasks, particularly for large NLP datasets, ensuring both reliable training and robust evaluation.

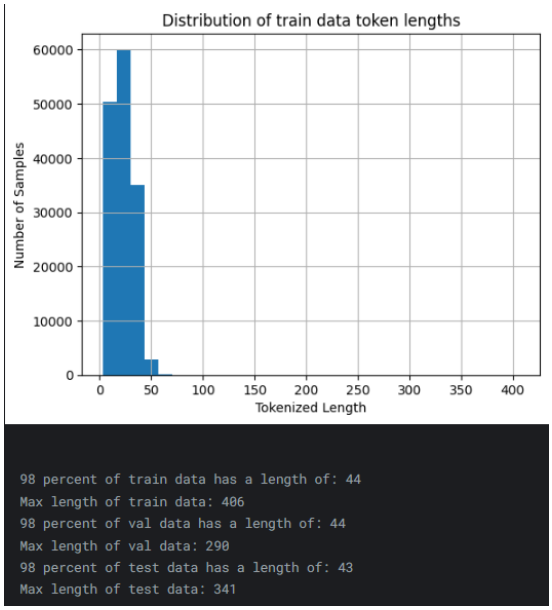
2.4. Vectorization

For the vectorization step, the pretrained tokenizers from the transformers library were used. Specifically, the BERT tokenizer for the BERT model and the DistilBERT tokenizer for the DistilBERT model. These tokenizers convert raw text into input IDs and attention masks suitable for transformer based architectures, while also handling the insertion of special tokens such as [CLS] and [SEP].

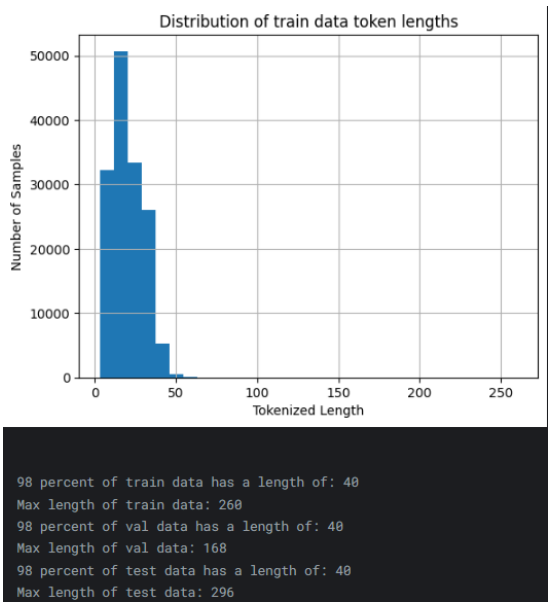
A crucial part of preparing input for transformer models is ensuring consistent input length across all samples. To achieve this, padding and truncation were applied. Padding ensures that all sequences reach a uniform length by adding special padding tokens, while truncation prevents sequences from exceeding the allowed input length. However, choosing this fixed input length (max_len) without prior analysis can result in unnecessary padding (if too large) or loss of important information (if too small).

To find an optimal balance, a custom analysis was performed on the training dataset to identify a suitable max_len value. The tokenized length of each training sample was computed, and the distribution of these lengths was analyzed. Instead of using the absolute maximum length, a percentile-based approach was adopted, where different percentile thresholds (e.g., 95th, 98th, 99th) were tested. The 98th percentile was found to provide the best trade-off between preserving most of the input content and minimizing excessive padding. This choice allowed the vast majority of sequences to be retained in full, while improving training speed and memory efficiency compared to padding all sequences to the maximum observed length. Additionally, it contributed to an increase in overall model accuracy, as more representative and efficiently processed input sequences led to better learning.

The following figure shows the distribution of tokenized sequence lengths in the training dataset, which guided the selection of the final max_len value.



(a) BERT token distribution



(b) DistilBERT token distribution

3. Algorithms and Experiments

3.1. Experiments

Building an effective classification model requires iterative experimentation to refine its performance. This section details the approach taken to optimize the models, starting with an initial baseline run followed by systematic improvements through various techniques. The goal was to identify methods that enhance generalization, improve accuracy, and mitigate overfitting.

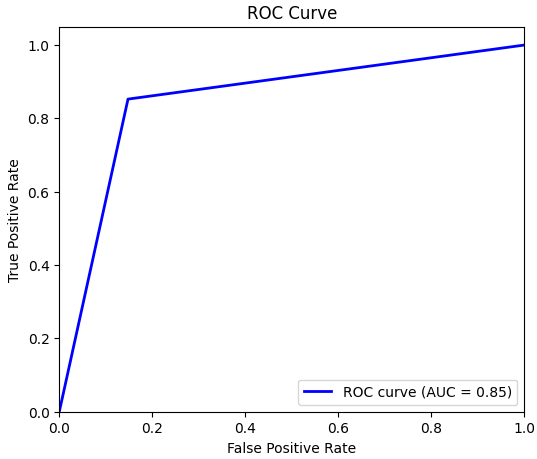
As in the previous projects, a large number of experiments were conducted throughout the fine tuning process. Rather than presenting each one in detail, this report focuses on a selection of representative experiments that highlight the impact of various optimization techniques. These include both successful and unsuccessful attempts, as understanding what didn't work is just as important as what did. Each experiment is evaluated using a range of metrics, including learning curves, confusion matrices, ROC curves, and classification reports, to provide a comprehensive view of model performance.

For each experiment, the results are compared between the two models to assess how each architecture responds to the applied optimizations and to better understand their respective strengths and weaknesses.

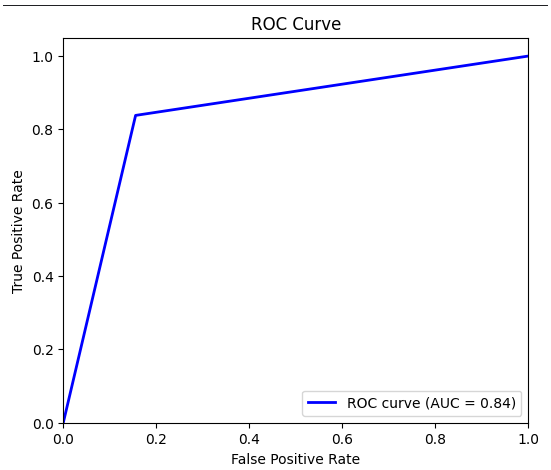
3.1.1. Experiment 1 - Baseline models. The first experiment focuses on fine tuning two straightforward baseline models(BERT and DistilBERT) without applying any hyperparameter tuning or text preprocessing. Each model leverages its respective pretrained base version and uses the corresponding tokenizer, as described in the previous section. Despite the absence of preprocessing and optimization, both models demonstrated strong performance, clearly outperforming the results of the previous projects, which relied on traditional machine learning (TF-IDF with logistic regression) and a simple feedforward neural network architecture with Word2Vec embeddings.

This initial experiment serves as a reference point, providing insight into the dataset's learnability and the model's raw performance before applying any optimizations. By establishing this baseline, we can later compare the impact of different improvements such as text preprocessing, hyperparameter tuning, feature selection, and optimization techniques. The baseline models achieved an accuracy of 83.48% and 82.36% respectively on the training dataset and 85.24% and 84.13% on the validation dataset. While these results indicate that the models are already capturing some patterns in the data, there is still room for improvement.

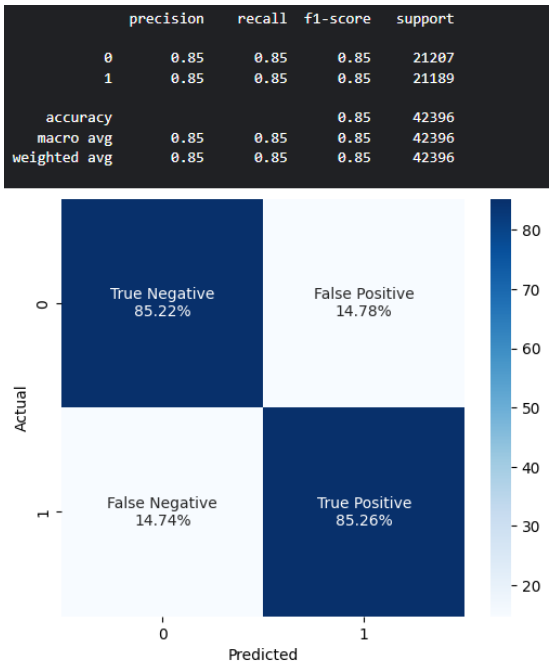
It is important to note that the models in this experiment were trained for only one epoch. Because of this, learning progress over time cannot be properly observed, and learning curves are not included, as a single epoch is not enough to show meaningful training behavior.



(a) BERT ROC Curve, Experiment 1

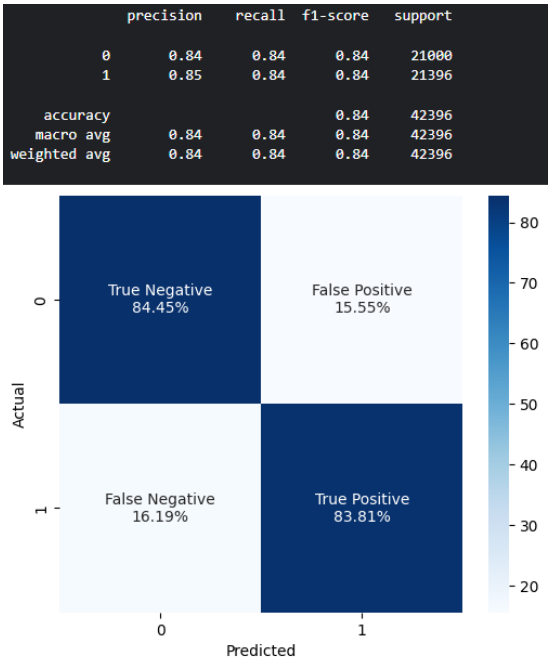


(b) DistilBERT ROC Curve, Experiment 1



	precision	recall	f1-score	support
0	0.85	0.85	0.85	21207
1	0.85	0.85	0.85	21189
accuracy			0.85	42396
macro avg	0.85	0.85	0.85	42396
weighted avg	0.85	0.85	0.85	42396

(a) BERT Classification Report and Confusion Matrix, Experiment 1



	precision	recall	f1-score	support
0	0.84	0.84	0.84	21000
1	0.85	0.84	0.84	21396
accuracy			0.84	42396
macro avg	0.84	0.84	0.84	42396
weighted avg	0.84	0.84	0.84	42396

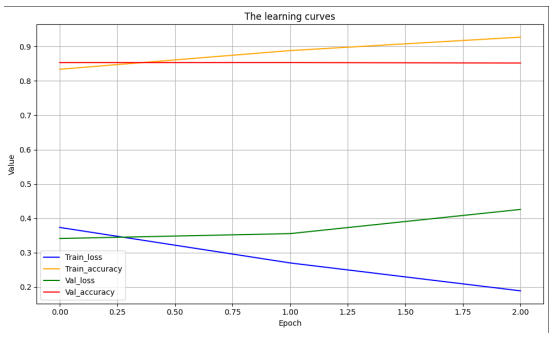
(b) DistilBERT Classification Report and Confusion Matrix, Experiment 1

3.1.2. Experiment 2 - Number of epochs. In Experiment 2, the focus was on investigating how the number of training epochs affects model performance. Each model was trained for up to three epochs to examine whether performance improves or deteriorates with extended training. Additionally, the full text preprocessing pipeline, as described in the corresponding section, was introduced to evaluate its impact.

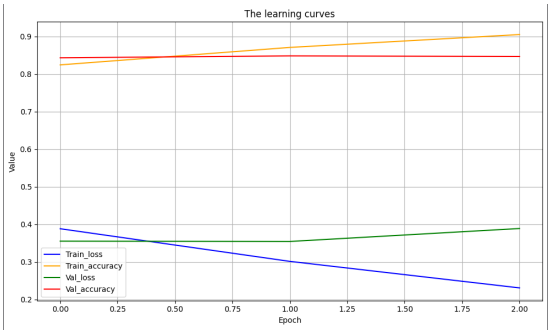
Training for too few epochs may result in the model not learning enough from the data, while training for too many epochs may lead to overfitting, where the model memorizes the training data and performs poorly on new, unseen data. The aim was to find a balance between learning enough patterns from the training data and maintaining good generalization on the validation set.

The results indicate that increasing the number of epochs led to a drop in performance for both models, suggesting early signs of overfitting. Although more training

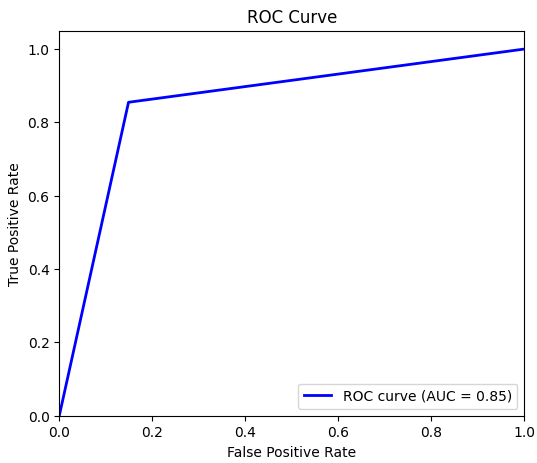
might intuitively seem beneficial, these outcomes highlight the importance of finding the right balance between sufficient learning and maintaining generalization, especially when working with powerful transformer based models. Specifically, the BERT model achieved 92.75% accuracy on the training set and 85.22% on the validation set, while the DistilBERT model reached 90.51% on training and 84.67% on validation. Compared to the second epoch, these results show only marginal gains in training accuracy but a slight drop in validation performance, reinforcing the presence of overfitting.



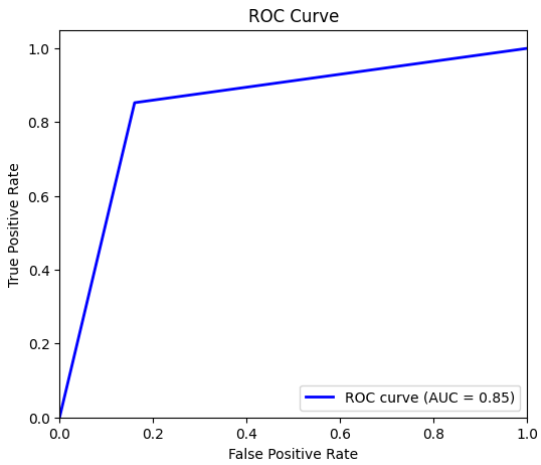
(a) BERT Learning Curve, Experiment 2



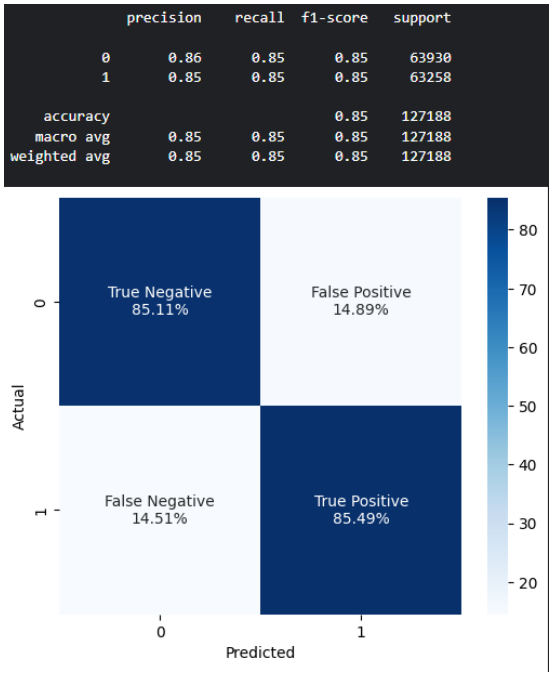
(b) DistilBERT Learning Curve, Experiment 2



(a) BERT ROC Curve, Experiment 2

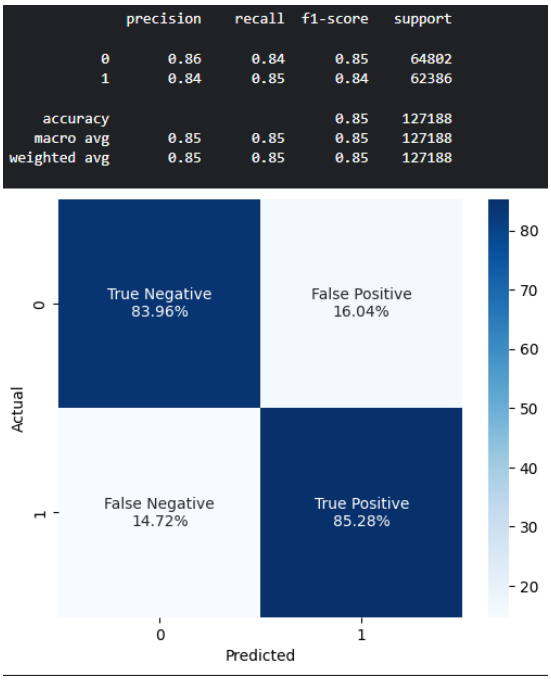


(b) DistilBERT ROC Curve, Experiment 2



	precision	recall	f1-score	support
0	0.86	0.85	0.85	63930
1	0.85	0.85	0.85	63258
accuracy			0.85	127188
macro avg	0.85	0.85	0.85	127188
weighted avg	0.85	0.85	0.85	127188

(a) BERT Classification Report and Confusion Matrix, Experiment 2



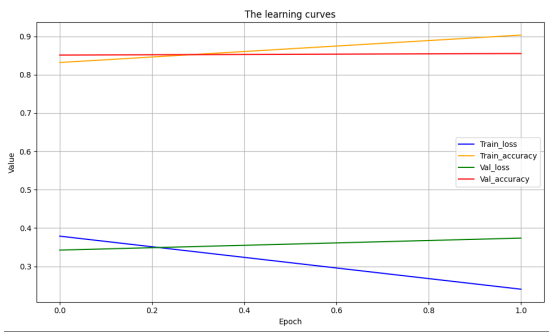
	precision	recall	f1-score	support
0	0.86	0.84	0.85	64802
1	0.84	0.85	0.84	62386
accuracy			0.85	127188
macro avg	0.85	0.85	0.85	127188
weighted avg	0.85	0.85	0.85	127188

(b) DistilBERT Classification Report and Confusion Matrix, Experiment 2

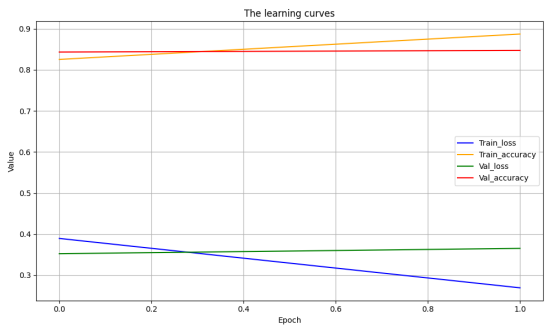
3.1.3. Experiment 3 - Learning rate. The third experiment aims to analyze the impact of the learning rate on the training dynamics and overall performance of the two models. This hyperparameter plays a crucial role in fine tuning transformer based models, as it directly affects how quickly and effectively the model converges to a minimum of the loss function.

In this experiment, each model was trained using a learning rate of 5e-5, which is commonly used as a baseline in transformer fine-tuning tasks. The results indicate that this value provided stable training and solid performance for both models, without signs of divergence or underfitting.

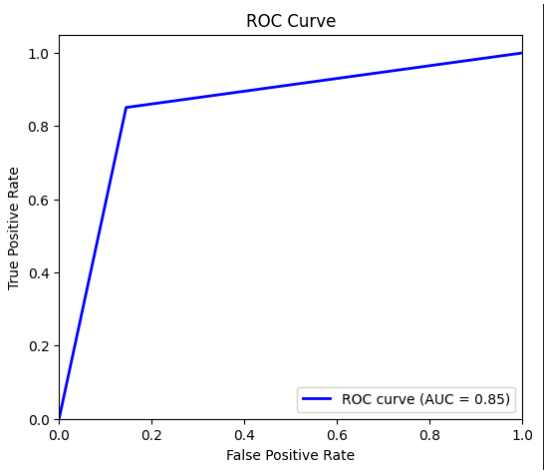
The BERT model achieved an accuracy of 90.33% on the training set and 85.51% on the validation set, while DistilBERT reached 88.72% and 84.73% respectively.



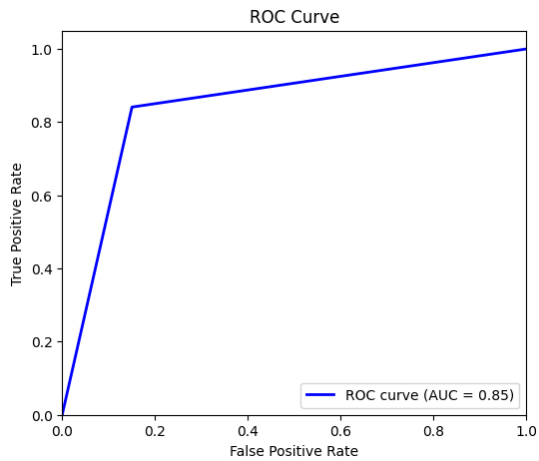
(a) BERT Learning Curve, Experiment 3



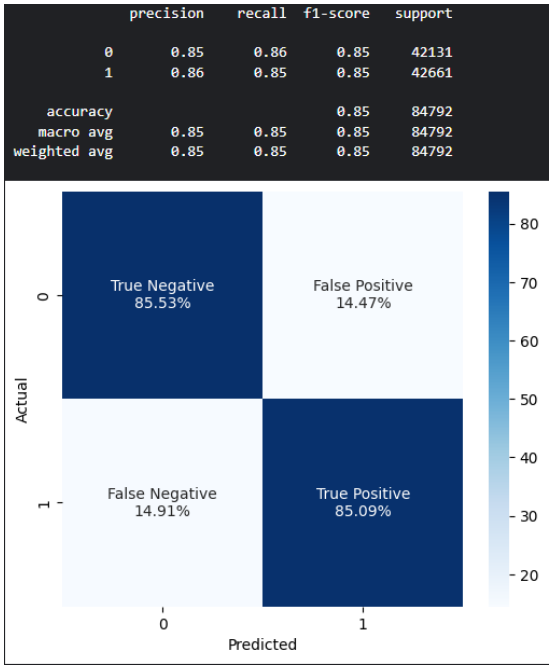
(b) DistilBERT Learning Curve, Experiment 3



(a) BERT ROC Curve, Experiment 3

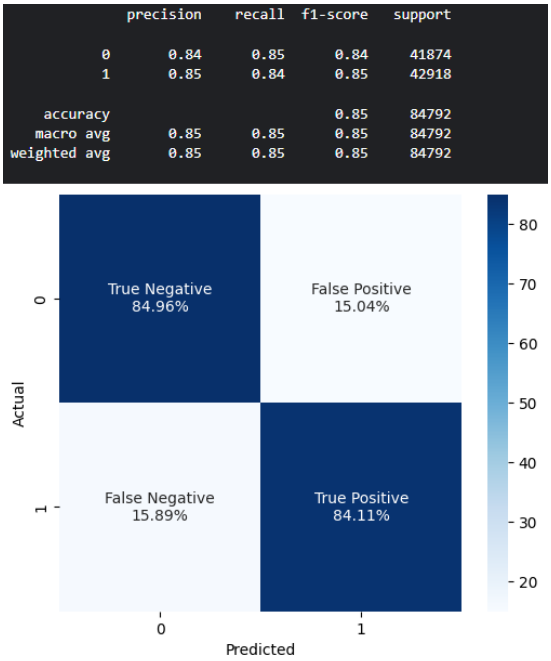


(b) DistilBERT ROC Curve, Experiment 3



	precision	recall	f1-score	support
0	0.85	0.86	0.85	42131
1	0.86	0.85	0.85	42661
accuracy			0.85	84792
macro avg	0.85	0.85	0.85	84792
weighted avg	0.85	0.85	0.85	84792

(a) BERT Classification Report and Confusion Matrix, Experiment 3



	precision	recall	f1-score	support
0	0.84	0.85	0.84	41874
1	0.85	0.84	0.85	42918
accuracy			0.85	84792
macro avg	0.85	0.85	0.85	84792
weighted avg	0.85	0.85	0.85	84792

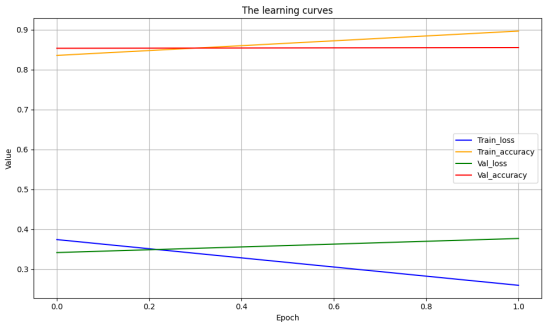
(b) DistilBERT Classification Report and Confusion Matrix, Experiment 3

3.1.4. Experiment 4 - Batch size. The fourth experiment aims to investigate the impact of batch size on the models' performance in terms of both training time and evaluation metrics. Batch size is an important hyperparameter in deep learning, as it determines how many training samples are processed before the model updates its weights. A smaller batch size may lead to more stable generalization but slower training, while a larger batch size can speed up training but sometimes at the cost of poorer generalization or convergence instability.

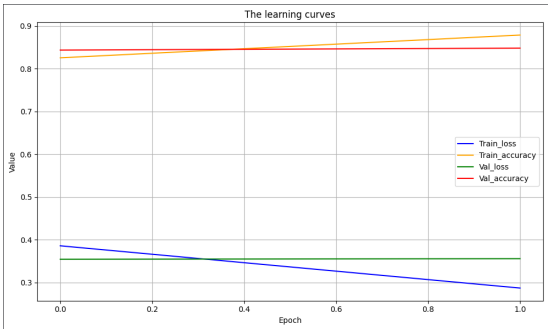
In this experiment, both models were trained using a batch size of 16, which is a commonly used trade off that balances memory usage and convergence behavior.

The results of this experiment showed that both models maintained strong performance, but the training time was significantly longer compared to previous experiments. BERT achieved an accuracy of 89.65% on the training set and 85.52% on the

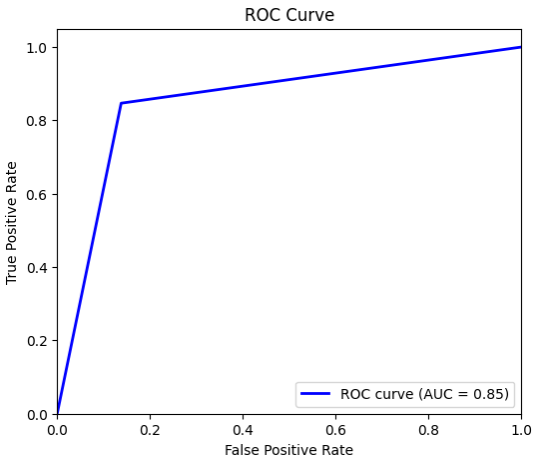
validation set, while DistilBERT scored 87.88% and 84.82% respectively. However, the performance improvement was marginal, indicating that increasing the batch size to 16 did not lead to substantial gains in accuracy. This suggests that, although a batch size of 16 is a safe and commonly used choice, it may not offer a good trade-off between training time and performance for this specific task and setup.



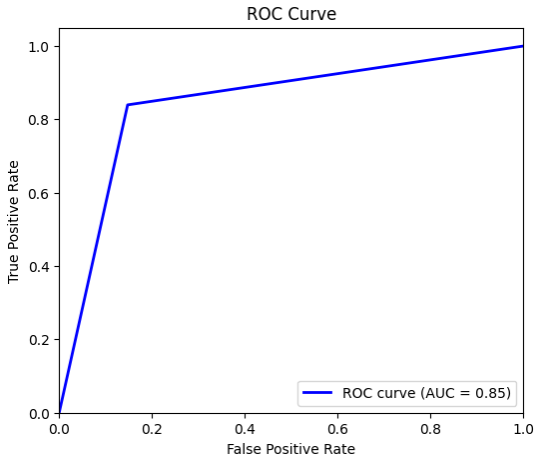
(a) BERT Learning Curve, Experiment 4



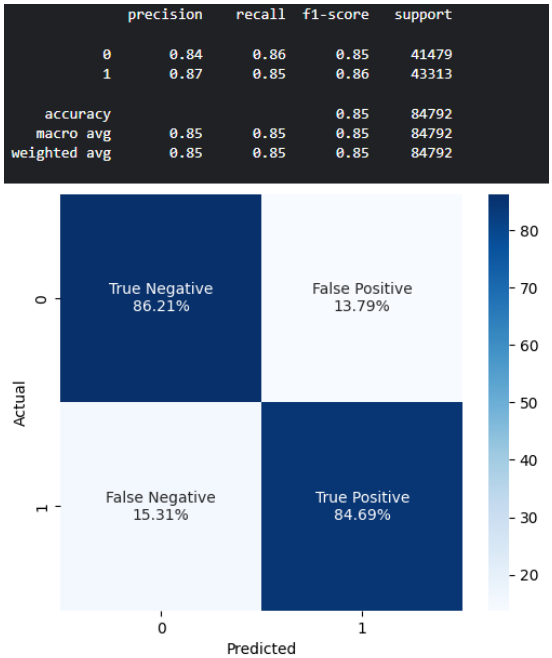
(b) DistilBERT Learning Curve, Experiment 4



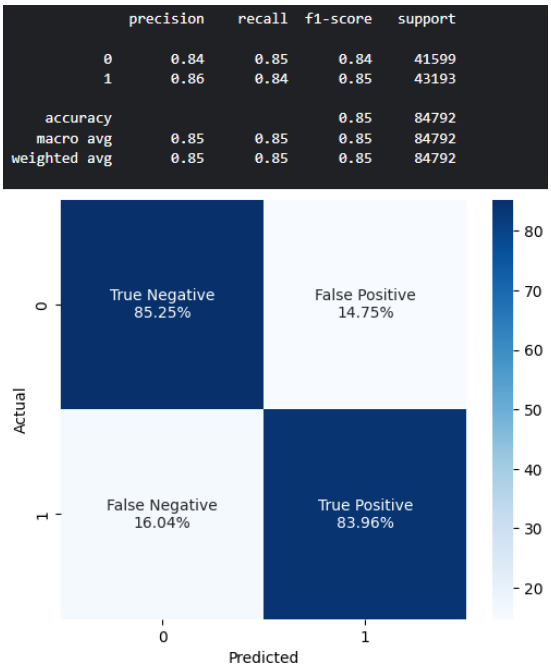
(a) BERT ROC Curve, Experiment 4



(b) DistilBERT ROC Curve, Experiment 4



(a) BERT Classification Report and Confusion Matrix, Experiment 2



(b) DistilBERT Classification Report and Confusion Matrix, Experiment 2

3.1.5. Table of trials. The following table summarizes the different experiments conducted, including variations in text preprocessing, hyperparameters, and resulting performance metrics.

Trial	TP	EP	LR	BS	TA	VA
E1	No	1	2e-5	32	83.48%	85.24%
E2	Yes	3	2e-5	32	92.75%	85.22%
E3	Yes	2	5e-5	32	90.33%	85.51%
E4	Yes	2	2e-5	16	89.65%	85.52%

Table 1: Summary of the trials conducted and their results for BERT

Trial	TP	EP	LR	BS	TA	VA
E1	No	1	2e-5	32	82.36%	84,13%
E2	Yes	3	2e-5	32	90.51%	84.67%
E3	Yes	2	5e-5	32	88.72%	84.73%
E4	Yes	2	512	5	87.88%	84.82%

Table 2: Summary of the trials conducted and their results for DistilBERT

*The "E" in the trials stands for Experiment, TP stands for Text Preprocessing, EP stands for Epochs, LR stands for Learning Rate, BS stands for batch size,TA stands for Training Accuracy and VA stands for Validation Accuracy

3.2. Hyper-parameter tuning

Hyperparameter tuning is a crucial step in optimizing the models performance, as it directly impacts both their ability to learn from data and their generalization to unseen samples. This section outlines the most important hyperparameters and presents the final selection of the optimal values.

3.2.1. Key Hyperparameters. Fine tuning BERT and DistilBERT is generally more straightforward compared to training a model from scratch, as it involves fewer parameters to adjust. During the tuning process, the most critical hyperparameters considered were the following:

- **Hidden layers**: Refers to the number of fully connected layers added on top of the pretrained model for classification. Increasing them can help the model capture more task specific patterns, but also increases complexity and the risk of overfitting.
- **Batch size**: This determines how many samples are used to calculate and update the model's weights in each training step. Smaller batch sizes can help generalization, while larger ones can make training more stable and faster on some hardware.
- **Loss function**: The loss function measures how far the model's predictions are from the correct values. It is used to guide the learning process.
- **Optimizer**: The optimizer decides how the model's weights are updated during training. Popular choices like Adam or SGD have different strengths depending on the task and data.
- **Learning rate**: This controls how big the updates to the model's weights are. A learning rate that is too high might make the model unstable, while one that is too low can make training very slow.
- **Epochs**: The number of times the model goes through the entire training dataset. More epochs allow the model to learn better, but too many may lead to overfitting.
- **Dropout rate**: A regularization method where a fraction of neurons is randomly deactivated during training. This helps prevent overfitting and improves the model's ability to generalize to unseen data.
- **Scheduler**: Adjusts the learning rate during training based on a predefined strategy. This helps the model converge more efficiently by using higher learning rates in the beginning and gradually reducing them, often leading to better generalization and stability.

3.2.2. Hyperparameters Tuning Results. The results of the hyperparameter tuning process can be grouped into two main categories: parameters with an easier optimal choice based on prior knowledge, and parameters that required more extensive experimentation to fine tune.

In the first category we can place the loss function, the optimizer, and the scheduler. For the optimizer, AdamW was selected, as it is specifically recommended for

transformer based models and is also the default choice in the Hugging Face library. For the loss function, Cross Entropy Loss was used, which is suitable for binary classification tasks when the labels are provided in categorical format. Regarding the scheduler, the Linear Scheduler without warmup was chosen after testing alternative schedulers(such as cosine or step-based decay) and warmup strategies, which did not show any consistent improvement in model performance. Given the simplicity of the task, the linear scheduler provided a good balance between stability and simplicity. These choices, AdamW optimizer, Cross Entropy Loss and the Linear Scheduler without warmup were applied consistently to both BERT and DistilBERT models.

In the second category, parameters such as the number of hidden layers (in the classification head), batch size, learning rate, number of epochs, and dropout rate were fine-tuned through systematic experimentation. Specifically, after extensive testing and taking into account the recommendations from the original BERT and DistilBERT authors, the following values were selected for each model to achieve the best balance between performance, training stability, and computational efficiency.

- **Hidden layers**: A single linear layer was used for both models as part of the classification head. More complex alternatives (e.g., multiple layers with activation functions) were consistently found to reduce accuracy.
- **Dropout rate**: No dropout was applied in the BERT model, as it led to a decrease in accuracy. For DistilBERT, a dropout rate of 0.1 was introduced, which slightly improved performance.
- **Batch size**: A batch size of 32 was selected for both models as it provided the best training speed and model accuracy.
- **Learning rate**: Both models performed best with a learning rate of 2e-5, yielding improved results on both the validation and test sets.
- **Number of Epochs**: Two epochs were chosen for both models. After the second epoch, signs of overfitting appeared, and accuracy began to drop.

This combination of hyperparameters was determined to be the most effective for the specific dataset and task, with consistent improvements in validation accuracy and overall model robustness.

3.2.3. Prevention of overfitting and underfitting. Through the above mentioned hyperparameters, the models were able to achieve a good balance between overfitting and underfitting, resulting in stable performance across different datasets. By carefully tuning the key parameters, the models retained enough complexity to capture meaningful patterns without being overly sensitive to noise. This approach allowed for robust models capable of generalizing well. Both overfitting and underfitting will be discussed in more detail and analyzed in the upcoming evaluation section.

3.3. Optimization techniques

The optimization strategy in this project focused on identifying the most effective combination of hyperparameters to ensure strong predictive performance and robust

generalization to unseen data. Hyperparameter tuning plays a critical role in transformer based models, as even small changes can significantly affect learning dynamics, convergence speed, and final model accuracy. Therefore, a careful and methodical approach was taken to balance training efficiency with model performance, while also considering the available computational resources.

Unlike the previous project, which combined manual tuning with automated optimization via Optuna, this time the entire process was carried out manually. While Optuna was initially considered, it was ultimately set aside due to the computational demands of fine tuning large transformer based models like BERT and DistilBERT. The long training times made extensive hyperparameter search impractical.

Moreover, using Optuna on only a subset of the dataset was deemed unsuitable, as it could potentially obscure important data patterns from the optimization process, leading to misleading results. Manual tuning, while more time consuming, allowed for a more informed and controlled exploration of the hyperparameter space, tailored to the specific behavior of each model during training.

3.4. Evaluation

Evaluating the model's performance is a crucial step in understanding its effectiveness in classification tasks. To assess the quality of predictions, multiple evaluation metrics and visualizations are used. These metrics help determine whether the model generalizes well to unseen data, suffers from overfitting, or struggles with class imbalances.
The evaluation process includes:

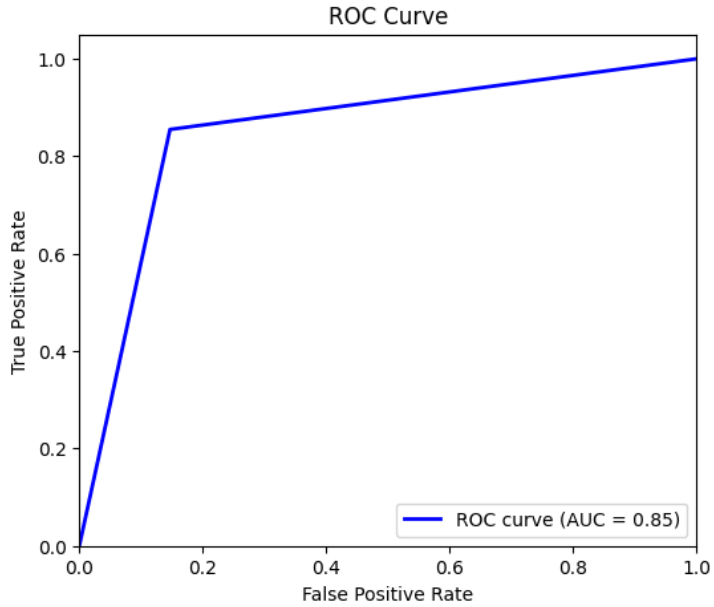
- **ROC Curve (Receiver Operating Characteristic Curve)**: Measures the model's ability to distinguish between classes across different threshold values.
- **Learning Curve**: Tracks the model's performance over time by plotting the training and validation loss and accuracy. It provides insights into the model's progress during training, helping to detect overfitting or underfitting.
- **Classification Report**: Provides a detailed breakdown of key classification metrics such as precision, recall, and F1-score.
- **Confusion Matrix**: Gives insights into the model's misclassifications by showing the distribution of true positives, false positives, true negatives, and false negatives.

Each of these evaluation tools plays a key role in analyzing the strengths and weaknesses of each model. In the following subsections, these metrics and visualizations will be discussed in detail, along with their respective results and interpretations.

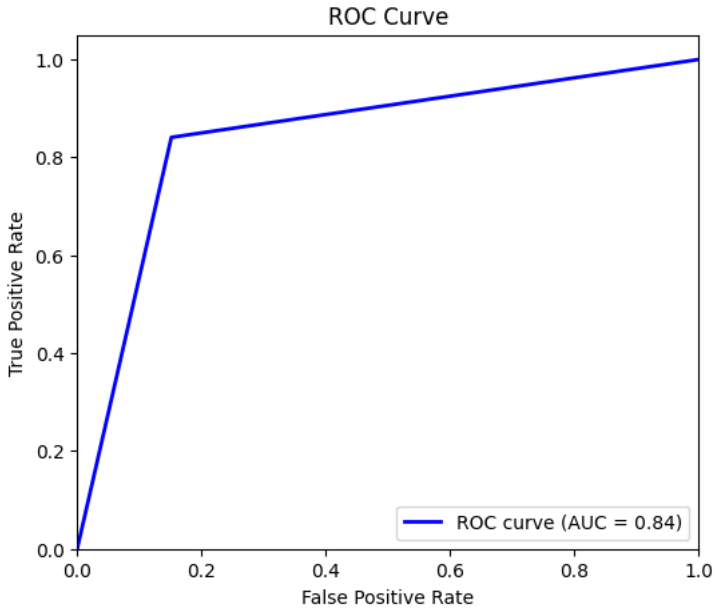
3.4.1. ROC curve. The Receiver Operating Characteristic (ROC) curve is a fundamental evaluation tool for assessing the performance of a classification model. It illustrates the trade off between True Positive Rate (TPR) and False Positive Rate (FPR) across different decision thresholds. A well performing model will have a curve that approaches the top left corner, indicating a high true positive rate with a low false positive rate.

To quantify the model's ability to distinguish between classes, the Area Under the Curve (AUC-ROC) is used. A value of 1.0 represents a perfect classifier, whereas 0.5 indicates a model that is no better than random guessing.

The ROC curve for each model is presented, illustrating their performance in distinguishing between the two classes. The AUC score achieved by BERT was 0.85, and by DistilBERT was 0.84, reflecting a solid discriminative ability. Both models exhibit a sharp initial rise in the curve, indicating their effectiveness in distinguishing classes with lower false positive rates. As the threshold increases, the curve shows a more gradual slope, which suggests that the models continue to perform well but with slightly less confidence as the decision threshold becomes stricter.



BERT ROC Curve



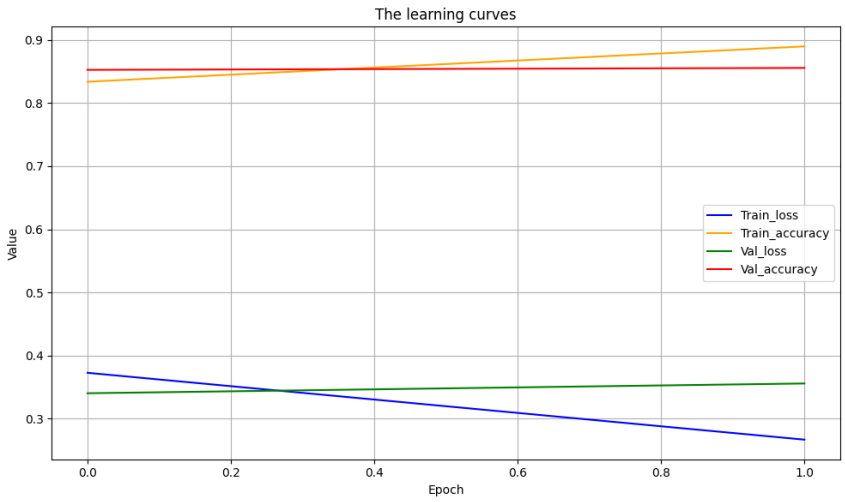
DistilBERT ROC Curve

3.4.2. Learning Curve. The learning curves of the models are presented to illustrate how the training and validation loss evolved over time across epochs. This visualization provides valuable insight into how well the models learned from the training data and how effectively they generalized to unseen validation data.

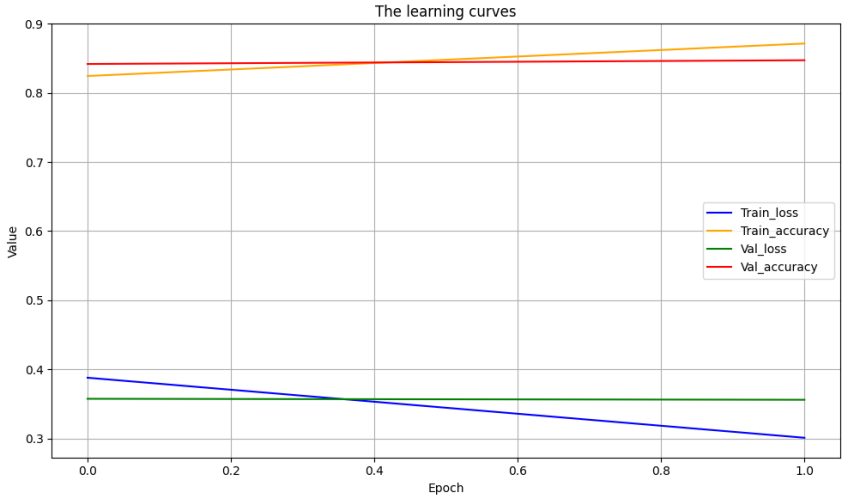
Throughout training, the training loss showed a steady and consistent decrease, while the validation loss followed a similar downward trend with only a slight increase towards the end. Despite this minor increase, no overfitting was observed as the models accuracy continued to improve, demonstrating that they were still able to generalize effectively to unseen data.

Importantly, no signs of overfitting or underfitting were observed. The validation loss increased only marginally, which did not affect the accuracy, indicating that the model maintained its ability to generalize well. In cases of overfitting, we would expect the validation loss to increase more significantly, while the training loss would continue to decrease. On the other hand, if the models were underfitting, both losses would remain high or fluctuate unpredictably. Neither of these issues occurred, confirming that the training process was well balanced.

Overall, the learning curve demonstrates that the models achieved stable and generalizable learning, without underfitting or overfitting. This reflects a well tuned architecture and an effective optimization process.



BERT Learning Curve



DistilBERT Learning Curve

3.4.3. Classification Report. The classification report of the model presents key performance metrics, including precision, recall, and F1-score for both classes.

Both models achieved strong and balanced predictive performance on the dataset, with BERT reaching an overall accuracy of 85% and DistilBERT achieving 84%. These results indicate that both models generalize well to unseen data, without signs of overfitting or underfitting.

For BERT, the classification report shows:

- **Class 0:** precision of 0.86, recall of 0.85, and an F1-score of 0.85. This means that when BERT predicts class 0, it's correct 86% of the time and successfully identifies 85% of actual class 0 instances.
- **Class 1:** precision of 0.85, recall of 0.86, and an F1-score of 0.85.
- The **macro average F1-score** is 0.85, and the weighted average F1-score is also 0.85.

	precision	recall	f1-score	support
0	0.86	0.85	0.85	42510
1	0.85	0.86	0.85	42282
accuracy			0.85	84792
macro avg	0.85	0.85	0.85	84792
weighted avg	0.85	0.85	0.85	84792

BERT Classification Report

For DistilBERT, the pattern is similar, though slightly lower:

- **Class 0:** precision of 0.84, recall of 0.85, and F1-score of 0.84.
- **Class 1:** precision of 0.85, recall of 0.84, and F1-score of 0.85.
- Both the **macro** and **weighted average F1-scores** are 0.84.

	precision	recall	f1-score	support
0	0.84	0.85	0.84	41924
1	0.85	0.84	0.85	42868
accuracy			0.84	84792
macro avg	0.84	0.84	0.84	84792
weighted avg	0.84	0.84	0.84	84792

DistilBERT Classification report

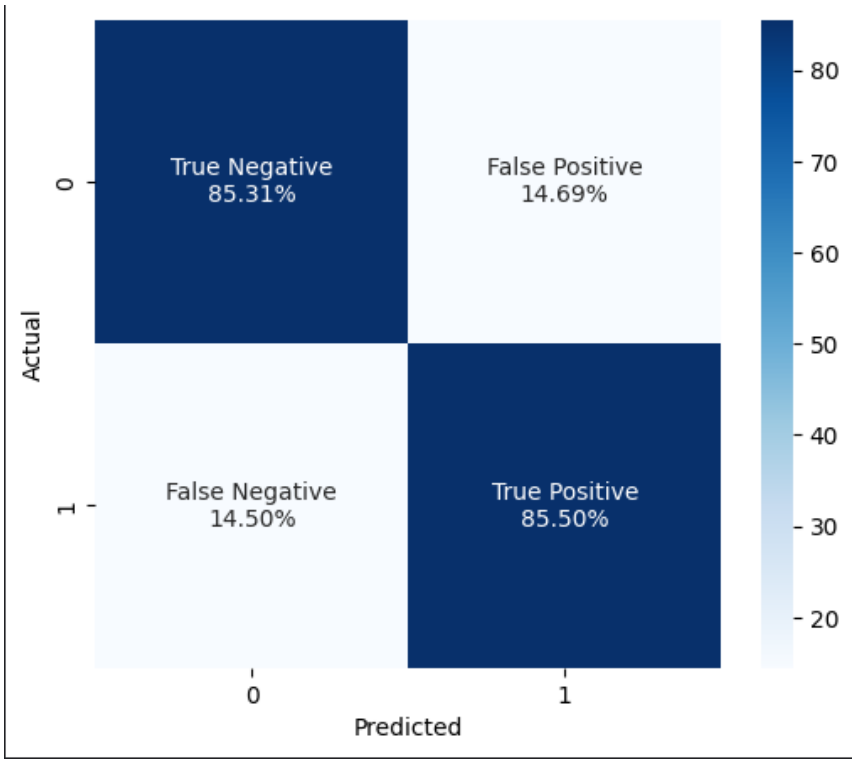
Overall, both classification reports confirm that the models are well tuned and maintain solid performance across both classes. While BERT has a slight edge in overall accuracy and F1-scores, DistilBERT still delivers competitive results with a significantly smaller architecture.

3.4.4. Confusion matrix. The confusion matrices for both models offer a detailed view of how well each one performed on the classification task by showing correct and incorrect predictions for each class.

For BERT, the confusion matrix shows:

- **True Negatives (TN):** 85.31% of actual class 0 cases were correctly predicted as class 0.
- **False Positives (FP):** 14.69% of actual class 0 cases were incorrectly predicted as class 1.
- **False Negatives (FN):** 14.50% of actual class 1 cases were incorrectly predicted as class 0.
- **True Positives (TP):** 85.50% of actual class 1 cases were correctly predicted as class 1.

BERT shows slightly better recall for class 1 and maintains balanced performance across both classes. The error rates (FP and FN) are relatively low, suggesting that the model makes consistent and reliable predictions.

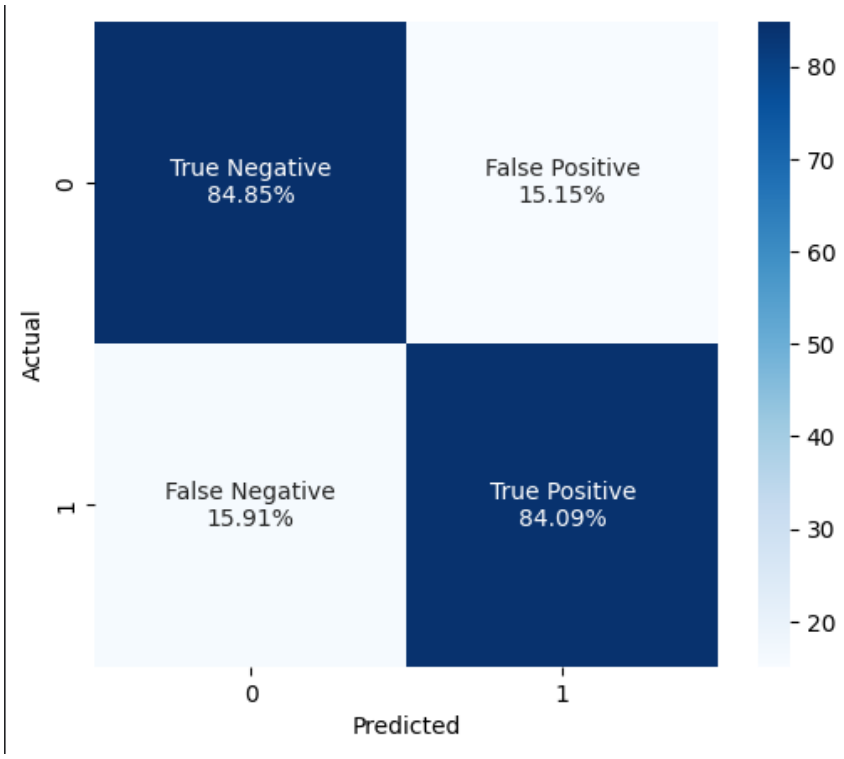


BERT Confusion Matrix

For DistilBERT, the confusion matrix shows:

- **True Negatives (TN):** 84.85% of actual class 0 cases were correctly predicted as class 0.
- **False Positives (FP):** 15.15% of actual class 0 cases were incorrectly predicted as class 1.
- **False Negatives (FN):** 15.91% of actual class 1 cases were incorrectly predicted as class 0.
- **True Positives (TP):** 84.09% of actual class 1 cases were correctly predicted as class 1.

DistilBERT performs slightly lower than BERT but still maintains balanced classification across both classes. While its false positive and false negative rates are marginally higher, it remains a dependable and efficient model, especially given its smaller size and faster training time.



DistilBERT Confusion Matrix

In both cases, the structure of the confusion matrices confirms that the models are reliable and handle the classification task well without signs of overfitting or underfitting. They effectively distinguish between the two classes, with BERT performing slightly better overall.

4. Results and Overall Analysis

4.1. Results Analysis

Following the systematic development of both models, including thorough text preprocessing, feature engineering, and hyperparameter tuning the final results provide a clear picture of their performance.

For BERT, the ROC curve achieved an AUC score of 0.85, indicating a strong ability to differentiate between the two classes. The learning curve shows that the model was well trained, with both training and validation loss following a smooth, converging trajectory, suggesting no signs of underfitting or overfitting. The classification report reflects balanced precision, recall, and F1-score across both classes, confirming that the model maintains consistent and dependable performance. The confusion matrix further supports this, showing that the true positive and true negative rates are well-aligned, with relatively few misclassifications.

For DistilBERT, the AUC score reached 0.84, which also demonstrates good discriminative capability. While slightly lower than BERT, the performance remains competitive. The learning and validation losses evolved in a similarly stable pattern, and accuracy continued to improve even when validation loss plateaued slightly indicating generalization without overfitting. The classification metrics are also balanced, and

the confusion matrix highlights a solid distribution of correct predictions across both classes, with minimal error rates.

Although the results are strong, further experiments could be conducted to explore potential improvements:

- **Experimentation with alternative tokenization or embedding strategies**: Investigating the effect of different tokenization methods (e.g., byte-level BPE) or richer input representations such as character level embeddings or contextualized embeddings from domain specific models could provide more informative input to the transformer and improve performance.
- **Adoption of more advanced transformer architectures**: Exploring models such as RoBERTa, XLNet or DeBERTa, which are known to outperform BERT on various NLP tasks, could lead to better classification results.
- **Data Number**: Increasing the dataset size by collecting more labeled examples or performing data augmentation could enhance the models ability to generalize and reduce potential biases.

4.1.1. Best trial. The best trials achieved in this project reflect a careful balance between strong performance metrics and effective generalization without signs of overfitting. The process for reaching these results involved consistent preprocessing, thoughtful vectorization through pre trained transformers, and extensive hyperparameter tuning, as thoroughly explained in earlier sections and supported by evaluation plots.

After applying these steps, the best configurations for each model are summarized below:

- **Text Preprocessing:** The text preprocessing , as described in Section 2.1, ensured that the data was cleaned and transformed effectively before feature extraction
- **Vectorization:** As mentioned in section 2.4, both BERT and DistilBERT used their internal pre trained contextual embeddings through their default tokenizers. Padding and truncation were applied to 98% of the dataset length to ensure uniform input size while avoiding excessive loss of information. This strategy preserved most of the textual content while maintaining computational efficiency.
- **Classification Head:** For both models, a simple classification head was used, consisting of a single linear layer. This choice was informed by experimental evidence, where more complex heads resulted in reduced accuracy.
- **Training Configuration:**
  - **BERT:** Used a dropout rate of 0, batch size of 32, learning rate of 2e-5, and was trained for 2 epochs using the AdamW optimizer and Cross Entropy Loss. A linear scheduler without warmup was applied.
  - **DistilBERT:** Used a dropout rate of 0.1, batch size of 32, learning rate of 2e-5, and was trained for 2 epochs with the same optimizer, loss function, and scheduler as BERT.

- **Overfitting and Underfitting Prevention:** Model evaluation(Section 3.4) confirmed that both models maintained a generalizable performance. While the validation loss increased slightly, the validation accuracy continued to rise, indicating no overfitting.
- **Model Evaluation:** Both models were evaluated using multiple metrics including accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. These evaluations, detailed in Section 3.4, confirmed stable and balanced performance across classes.

In conclusion, the final results were as follows:

- **BERT:** Achieved an accuracy of 88.97% on the training set, 85.56% on the validation set and 85.678% on the test set.
- **DistilBERT:** Achieved an accuracy of 87.16% on the training set, 84.74% on the validation set and 85.027% on the test set

The difference in validation and test accuracy between the two models is approximately 0.5% to 1%, indicating a comparable level of generalization performance. This small gap is expected and reflects the trade off between model size and predictive power.

Additionally, training DistilBERT was significantly faster, approximately half an hour compared to over an hour for BERT, highlighting its efficiency in resource constrained environments. Therefore, BERT is the preferred choice when slightly higher accuracy is desired, whereas DistilBERT is more suitable when faster training time and reduced computational requirements are prioritized.

4.2. Comparison with the First Project

Compared to the first project, which employed a traditional machine learning pipeline using TF-IDF vectorization and logistic regression, both transformer based models (BERT and DistilBERT) demonstrated a substantial improvement in performance. Specifically, the best accuracy achieved in the first project was approximately 80-81%, whereas in this project both BERT and DistilBERT exceeded 85% accuracy on the validation and test sets. This marks a significant performance gain of around 4% to 5%, indicating the superior capability of transformer models capture contextual and semantic details in text data.

However, this improvement comes at a considerable computational cost. While the TF-IDF model required only a few minutes (around 10 minutes) to train and evaluate on a standard CPU, the BERT and DistilBERT models required significantly more resources. Training was performed on two GPUs, and even with such hardware, the total training time was approximately 1 hour for BERT and about 30 minutes for DistilBERT. Moreover, the memory requirements and execution time during both training and inference were significantly higher for the transformer models.

In summary, while the adoption of BERT based models offers a notable boost in predictive accuracy and other metrics, it also introduces a much higher computational cost, which must be carefully considered depending on the use case and available resources.

4.3. Comparison with the second project

The second project involved a feed forward neural network (FFNN) utilizing Word2Vec embeddings for text representation. The results of this approach were somewhat similar to the first project, as the FFNN achieved an accuracy of approximately 80%, which is very close to the performance of the TF-IDF and logistic regression model.

However, in terms of computational cost, the FFNN required a GPU and took a longer time to execute compared to the TF-IDF approach. Specifically, the training time for the FFNN was higher than that of the TF-IDF model, but still significantly lower than that of BERT and DistilBERT, which required two GPUs and took around one and a half hours combined to complete.

In conclusion, while the FFNN achieved a similar accuracy to the TF-IDF model, it did not match the performance of the transformer based models (BERT and DistilBERT) in terms of accuracy and other metrics. However, its computational efficiency was better than the transformer models, with a notably shorter training time.

5. Bibliography

References

[1] Hugging Face Community. What is the purpose of the additional dense layer in classification heads?, 2022. URL.

[2] GeeksforGeeks. How to use hugging face pretrained model, 2023. URL.

[3] Amit Hey. Fine-tuning bert for classification – a practical guide, 2023. URL.

[4] Amit Hey. Fine-tuning neural network – a practical guide, 2023. URL.

[5] Hugging Face. Bert base uncased, 2024. URL.

[6] Hugging Face. Distilbert base uncased, 2024. URL.

[7] Manolis Koumparakis. Bert, 2025. URL.

[8] Manolis Koumparakis. Transformers, 2025. URL.

[9] Analytics India Magazine. A guide to text preprocessing using bert, 2023. URL.

[10] Despina-Athanasia Pantazi. Ai 2 tutorial – bert fine tuning, 2025. URL.

[11] Despina-Athanasia Pantazi. Ai 2 tutorial – bert word embeddings, 2025. URL.

[12] PyTorch. Pytorch, 2024. URL.

[13] Ashish Rao. Fine-tuning a pre-trained bert model for classification using native pytorch, 2022. URL.

[14] Researchify. Fine-tuning bert for text classification: A step-by-step guide with code examples, 2022. URL.

[15] Stanford University. Cs224n - lecture 5: Rnn language models, 2025. URL.

[16] Stanford University. Cs224n - lecture 6: Fancy rnns, 2025. URL.